

CYBER SECURITY INTERNSHIP REPORT

*Submitted in fulfilment of the 1-Month Cyber Security Internship
at Prodigy InfoTech*

Prepared by
Shaurya Parashar

Internship Domain	Cyber Security
Organization	Prodigy InfoTech
Duration	1st August 2026 – 31st August 2026 (1 Month)
Certificate ID (CIN)	PIT/AUG26/01162
Certificate Issued On	1st September 2026

www.prodigyinfotech.dev

Certificate of Completion

This report is submitted in support of the Certificate of Completion issued by Prodigy InfoTech, confirming successful completion of a one-month internship in Cyber Security with outstanding remarks. The certificate details are summarized below:

Awarded To	Shaurya Parashar
Domain	Cyber Security
Internship Period	1st August 2026 to 31st August 2026
Certificate Identification Number	PIT/AUG26/01162
Issued On	1st September 2026
Issuing Organization	Prodigy InfoTech (contact@prodigyinfotech.dev)

Acknowledgement

I would like to express my sincere gratitude to Prodigy InfoTech for providing me the opportunity to undertake this Cyber Security internship. This experience allowed me to strengthen my practical understanding of core security concepts such as cryptography, data protection, and system security through hands-on project work.

I am thankful to the mentors and the Prodigy InfoTech team for their guidance, structured task design, and continuous support throughout the internship period, which enabled me to complete all assigned tasks successfully and gain valuable industry-relevant experience.

About Prodigy InfoTech

Prodigy InfoTech is a technology-focused organization that offers virtual internship programs across domains such as Web Development, Android Development, Machine Learning, Data Science, and Cyber Security. The organization is registered under the Ministry of Micro, Small and Medium Enterprises (MSME), Government of India, and is known for structured, task-based internships that emphasize practical, self-driven learning.

The Cyber Security internship track is designed to introduce interns to foundational security concepts including classical and modern cryptography, data encryption techniques, authentication mechanisms, and awareness of offensive security tools, through a series of independently implemented Python projects.

Introduction

Cyber security has become one of the most critical domains in modern computing, as the volume and sensitivity of digital data continue to grow. This one-month internship with Prodigy InfoTech was undertaken to gain practical exposure to fundamental cyber security principles by designing and implementing small, self-contained Python applications that demonstrate encryption, data protection, authentication strength evaluation, and system monitoring concepts.

Over the course of the internship, four tasks were completed, each targeting a distinct area of cyber security: classical cryptography, file/image-level data encryption, password security auditing, and system-level input monitoring. Each task was implemented, tested, and verified using sample inputs, with terminal outputs confirming correct functionality.

Objectives of the Internship

- To understand and implement classical encryption techniques such as the Caesar Cipher.
- To explore file-level data protection by implementing an image encryption and decryption tool.
- To learn how password strength is evaluated and to build a rule-based password strength checker.
- To understand keystroke logging mechanisms and their security implications through an educational implementation.
- To gain hands-on programming experience in Python relevant to cyber security applications.

Tools and Technologies Used

- Programming Language: Python 3.11
- Development Environment: Visual Studio Code
- Libraries: tkinter (GUI), datetime (timestamping)
- Operating System: Windows
- File Format Studied: PPM (Portable Pixmap, P3 format) for image encryption

Task 1: Caesar Cipher Encryption & Decryption Tool

Objective

To implement one of the oldest known classical encryption techniques, the Caesar Cipher, and use it to encrypt and decrypt user-supplied text messages using a numeric shift key.

Description

The Caesar Cipher is a substitution cipher in which each letter of the plaintext is shifted a fixed number of positions down the alphabet. This task implements a `caesar()` function that iterates through each character of the input text, and for every alphabetic character, applies a modular shift of 26 (the size of

the English alphabet) to wrap the transformation correctly around 'z' back to 'a'. Non-alphabetic characters are left unchanged.

The same function is reused for both encryption and decryption: encryption applies a positive shift, while decryption applies the negative of the same shift, restoring the original message. The program takes the message and shift value as user input and prints both the encrypted and decrypted results to verify correctness.

Implementation (task1.py)

```
def caesar(text, shift):  
    result = ""  
  
    for char in text:  
        if char.isalpha():  
            result += chr((ord(char) - ord('a') + shift) % 26 + ord('a'))  
        else:  
            result += char  
  
    return result  
  
message = input("Enter message: ")  
shift = int(input("Enter shift: "))  
  
encrypted = caesar(message.lower(), shift)  
decrypted = caesar(encrypted, -shift)  
  
print("Encrypted:", encrypted)  
print("Decrypted:", decrypted)
```

Sample Output (Terminal)

```
Enter message: hello  
Enter shift: 3  
Encrypted: koor  
Decrypted: hello
```

Key Learning Outcomes

- Understood modular arithmetic as applied to substitution ciphers.
- Learned how a single reversible function can perform both encryption and decryption by sign-flipping the key.
- Practiced handling of case sensitivity and non-alphabetic characters in text-processing logic.

Task 2: Image Encryption & Decryption Tool

Objective

To design a tool capable of encrypting and decrypting image files by manipulating raw pixel data, demonstrating how data protection principles extend beyond plain text to binary/media files.

Description

This task works with images in the PPM (Portable Pixmap, ASCII P3) format, where an image file stores its header (format identifier, width, height, maximum colour value) followed by a flat list of RGB pixel values as plain numbers. The `encrypt()` function reads this data, extracts the header and pixel values, and applies a simple additive key-based transformation — each pixel value is shifted by adding a numeric key and reducing the result modulo 256, so every value stays within the valid 0–255 colour range.

The `decrypt()` function reverses this process by subtracting the same key (mod 256) from each pixel, restoring the original image data exactly, since addition and subtraction modulo 256 are inverse operations. Both functions preserve the original image header (width, height, and maximum colour value) so that the output remains a valid, viewable PPM image.

A menu-driven interface ties the two functions together: the user selects 'E' to encrypt or 'D' to decrypt, then provides the input image path, output image path, and the numeric key. This makes the tool usable as a simple command-line utility for protecting image files with a shared secret key.

Encryption Function (task2.py)

```
def encrypt(input_file, output_file, key):
    with open(input_file, "r") as f:
        data = f.read().split()

    width = int(data[1])
    height = int(data[2])
    max_value = int(data[3])

    pixels = list(map(int, data[4:]))

    for i in range(len(pixels)):
        pixels[i] = (pixels[i] + key) % 256

    with open(output_file, "w") as f:
        f.write("P3\n")
        f.write(f"{width} {height}\n")
        f.write(f"{max_value}\n")

    for pixel in pixels:
        f.write(str(pixel) + " ")
```

Decryption Function (task2.py)

```
def decrypt(input_file, output_file, key):
    with open(input_file, "r") as f:
        data = f.read().split()

    width = int(data[1])
    height = int(data[2])
    max_value = int(data[3])

    pixels = list(map(int, data[4:]))

    for i in range(len(pixels)):
        pixels[i] = (pixels[i] - key) % 256

    with open(output_file, "w") as f:
        f.write("P3\n")
        f.write(f"{width} {height}\n")
        f.write(f"{max_value}\n")

    for pixel in pixels:
        f.write(str(pixel) + " ")
```

Menu-Driven Interface (task2.py)

```
choice = input("Enter E for Encrypt or D for Decrypt: ").upper()
input_file = input("Enter input image: ")
output_file = input("Enter output image: ")
key = int(input("Enter key: "))

if choice == "E":
    encrypt(input_file, output_file, key)
    print("Image encrypted!")

elif choice == "D":
    decrypt(input_file, output_file, key)
    print("Image decrypted!")

else:
    print("Invalid choice!")
```

Sample Output (Terminal)

```
Enter E for Encrypt or D for Decrypt: D
Enter input image: encrypted.png
Enter output image: decrypted.png
Enter key: 50
Image decrypted!
```

Key Learning Outcomes

- Learned the structure of the PPM/P3 image format and how raw pixel data can be manipulated programmatically.
- Understood how modular arithmetic can be used to build a simple, symmetric-key style encryption scheme for binary/media data.
- Gained experience building a menu-driven command-line tool that combines multiple functions into a usable application.

Task 3: Password Strength Checker

Objective

To build a rule-based tool that evaluates the strength of a user-supplied password against common security best practices and provides instant feedback.

Description

Password security is one of the most fundamental lines of defence in cyber security. This task implements a scoring-based password strength checker that evaluates a password against five independent criteria: minimum length of 8 characters, presence of at least one uppercase letter, presence of at least one lowercase letter, presence of at least one digit, and presence of at least one special (non-alphanumeric) character.

Each satisfied criterion increments a running score by one, using Python's `any()` function combined with character-level checks such as `isupper()`, `islower()`, `isdigit()`, and `isalnum()`. Once all checks are complete, the final score (out of 5) is mapped to a strength label: a score of 5 is classified as 'Strong', a score of 3 or 4 as 'Medium', and any score below 3 as 'Weak'. This gives users immediate, actionable feedback on how to improve their password.

Implementation (task3.py)

```
password = input("Enter your password: ")

score = 0

# Check length
if len(password) >= 8:
    score += 1

# Check uppercase
if any(char.isupper() for char in password):
    score += 1

# Check lowercase
```

```

if any(char.islower() for char in password):
    score += 1

# Check number
if any(char.isdigit() for char in password):
    score += 1

# Check special character
if any(not char.isalnum() for char in password):
    score += 1

# Give feedback
if score == 5:
    print("Password Strength: Strong")
elif score >= 3:
    print("Password Strength: Medium")
else:
    print("Password Strength: Weak")

```

Key Learning Outcomes

- Understood the standard criteria used by real-world systems to enforce password complexity policies.
- Practiced using Python's `any()` function with generator expressions for concise character validation.
- Learned how a simple point-based scoring system can be used to translate multiple checks into a single, human-readable result.

Task 4: Simple Keylogger (Educational Purpose)

Objective

To build a basic GUI-based keystroke logger using Python's `tkinter` library, purely for educational purposes, in order to understand how keylogging software operates and why it represents a significant security and privacy risk.

Description

This task creates a small `tkinter` desktop application titled 'Simple Key Logger - Educational' containing a text box. A `key_pressed()` event handler function is bound to the window's `<KeyPress>` event, so it is triggered every time a key is pressed while the application window is in focus. The handler retrieves the symbolic name of the pressed key (`event.keysym`), displays it inside the on-screen text box, and simultaneously appends it — together with a timestamp generated using `datetime.now()` — to a local log file, `key_log.txt`.

It is important to note that this implementation deliberately listens only to keystrokes made while its own window is focused (via `window.bind()`), unlike malicious keyloggers which hook into system-wide keyboard events regardless of which application is active. This distinction was intentionally studied to understand the ethical and technical boundary between an educational demonstration and an actual malicious keylogging tool, and to appreciate why unauthorized system-wide keystroke capture is illegal and a serious privacy violation.

Implementation (task4.py)

```
from datetime import datetime

# Create window
window = tk.Tk()
window.title("Simple Key Logger - Educational")
window.geometry("500x300")

# Create text box
text_box = tk.Text(window, height=10, width=50)
text_box.pack(pady=20)

# File where keys will be saved
file_name = "key_log.txt"

def key_pressed(event):
    key = event.keysym

    # Show the key in the text box
    text_box.insert(tk.END, key + " ")

    # Save the key to the file
    with open(file_name, "a") as file:
        time = datetime.now().strftime("%H:%M:%S")
        file.write(f"{time} - {key}\n")

# Listen only to keys pressed while this application is focused
window.bind("<KeyPress>", key_pressed)
# Start program
window.mainloop()
```

Key Learning Outcomes

- Learned how GUI event binding works in tkinter, specifically the `<KeyPress>` event.
- Understood the technical mechanism behind keylogging software and why it poses a serious security and privacy threat.
- Learned the ethical distinction between a scoped, educational, in-app key logger and an unauthorized, system-wide malicious keylogger.
- Practiced file I/O for persistent, timestamped logging of runtime events.

Overall Skills Gained

- Practical understanding of classical and simple modern encryption/decryption techniques.
- Experience handling both plain text and binary/media (image) data for security applications.
- Ability to design rule-based evaluation systems, as demonstrated in the password strength checker.
- Foundational knowledge of GUI event handling and its relevance to security tools such as keyloggers.
- Improved proficiency in Python programming, file handling, and command-line application design.
- Awareness of ethical boundaries in cyber security tool development.

Conclusion

The one-month Cyber Security internship at Prodigy InfoTech provided valuable, hands-on exposure to core security concepts through four progressively insightful tasks — ranging from classical cryptography to media file encryption, password auditing, and system-level input monitoring. Each task was implemented, tested, and validated independently, reinforcing both theoretical understanding and practical programming skills in Python.

This internship has strengthened my foundation in cyber security fundamentals and has motivated me to continue exploring more advanced topics such as symmetric and asymmetric cryptography, network security, and ethical hacking. I am grateful to Prodigy InfoTech for this learning opportunity and the outstanding remarks received upon completion.

References

- Prodigy InfoTech — www.prodigyinfotech.dev
- Python 3 Official Documentation — docs.python.org
- Python tkinter Documentation — docs.python.org/3/library/tkinter.html
- Netpbm / PPM Image Format Specification — netpbm.sourceforge.net